

Initial results, though, show that it has the potential to outperform both standard Python and the Psyco extension in terms of execution speed, and it is a promising project. Search the Web for details on the project's current status.

Frozen Binaries

Sometimes when people ask for a “real” Python compiler, what they’re really seeking is simply a way to generate standalone binary executables from their Python programs. This is more a packaging and shipping idea than an execution-flow concept, but it’s somewhat related. With the help of third-party tools that you can fetch off the Web, it is possible to turn your Python programs into true executables, known as *frozen binaries* in the Python world.

Frozen binaries bundle together the byte code of your program files, along with the PVM (interpreter) and any Python support files your program needs, into a single package. There are some variations on this theme, but the end result can be a single binary executable program (e.g., an *.exe* file on Windows) that can easily be shipped to customers. In [Figure 2-2](#), it is as though the byte code and PVM are merged into a single component—a frozen binary file.

Today, three primary systems are capable of generating frozen binaries: *py2exe* (for Windows), *PyInstaller* (which is similar to *py2exe* but also works on Linux and Unix and is capable of generating self-installing binaries), and *freeze* (the original). You may have to fetch these tools separately from Python itself, but they are available free of charge. They are also constantly evolving, so consult <http://www.python.org> or your favorite web search engine for more on these tools. To give you an idea of the scope of these systems, *py2exe* can freeze standalone programs that use the *tkinter*, *PMW*, *wxPython*, and *PyGTK* GUI libraries; programs that use the *pygame* game programming toolkit; *win32com* client programs; and more.

Frozen binaries are not the same as the output of a true compiler—they run byte code through a virtual machine. Hence, apart from a possible startup improvement, frozen binaries run at the same speed as the original source files. Frozen binaries are not small (they contain a PVM), but by current standards they are not unusually large either. Because Python is embedded in the frozen binary, though, it does not have to be installed on the receiving end to run your program. Moreover, because your code is embedded in the frozen binary, it is more effectively hidden from recipients.

This single file-packaging scheme is especially appealing to developers of commercial software. For instance, a Python-coded user interface program based on the *tkinter* toolkit can be frozen into an executable file and shipped as a self-contained program on a CD or on the Web. End users do not need to install (or even have to know about) Python to run the shipped program.

Other Execution Options

Still other schemes for running Python programs have more focused goals:

- The *Stackless Python* system is a standard CPython implementation variant that does not save state on the C language call stack. This makes Python more easy to port to small stack architectures, provides efficient multiprocessing options, and fosters novel programming structures such as coroutines.
- The *Cython* system (based on work done by the *Pyrex* project) is a hybrid language that combines Python code with the ability to call C functions and use C type declarations for variables, parameters, and class attributes. Cython code can be compiled to C code that uses the Python/C API, which may then be compiled completely. Though not completely compatible with standard Python, Cython can be useful both for wrapping external C libraries and for coding efficient C extensions for Python.

For more details on these systems, search the Web for recent links.

Future Possibilities?

Finally, note that the runtime execution model sketched here is really an artifact of the current implementation of Python, not of the language itself. For instance, it's not impossible that a full, traditional compiler for translating Python source code to machine code may appear during the shelf life of this book (although one has not in nearly two decades!). New byte code formats and implementation variants may also be adopted in the future. For instance:

- The *Parrot* project aims to provide a common byte code format, virtual machine, and optimization techniques for a variety of programming languages (see <http://www.python.org>). Python's own PVM runs Python code more efficiently than Parrot, but it's unclear how Parrot will evolve.
- The *PyPy* project is an attempt to reimplement the PVM in Python itself to enable new implementation techniques. Its goal is to produce a fast and flexible implementation of Python.
- The Google-sponsored *Unladen Swallow* project aims to make standard Python faster by a factor of at least 5, and fast enough to replace the C language in many contexts. It is an optimization branch of CPython, intended to be fully compatible and significantly faster. This project also hopes to remove the Python multithreading Global Interpreter Lock (GIL), which prevents pure Python threads from truly overlapping in time. This is currently an emerging project being developed as open source by Google engineers; it is initially targeting Python 2.6, though 3.0 may acquire its changes too. Search Google for up-to-date details.

Although such future implementation schemes may alter the runtime structure of Python somewhat, it seems likely that the byte code compiler will still be the standard for

some time to come. The portability and runtime flexibility of byte code are important features of many Python systems. Moreover, adding type constraint declarations to support static compilation would break the flexibility, conciseness, simplicity, and overall spirit of Python coding. Due to Python's highly dynamic nature, any future implementation will likely retain many artifacts of the current PVM.

Chapter Summary

This chapter introduced the execution model of Python (how Python runs your programs) and explored some common variations on that model (just-in-time compilers and the like). Although you don't really need to come to grips with Python internals to write Python scripts, a passing acquaintance with this chapter's topics will help you truly understand how your programs run once you start coding them. In the next chapter, you'll start actually running some code of your own. First, though, here's the usual chapter quiz.

Test Your Knowledge: Quiz

1. What is the Python interpreter?
2. What is source code?
3. What is byte code?
4. What is the PVM?
5. Name two variations on Python's standard execution model.
6. How are CPython, Jython, and IronPython different?

Test Your Knowledge: Answers

1. The Python interpreter is a program that runs the Python programs you write.
2. Source code is the statements you write for your program—it consists of text in text files that normally end with a *.py* extension.
3. Byte code is the lower-level form of your program after Python compiles it. Python automatically stores byte code in files with a *.pyc* extension.
4. The PVM is the Python Virtual Machine—the runtime engine of Python that interprets your compiled byte code.
5. Psyco, Shedskin, and frozen binaries are all variations on the execution model.
6. CPython is the standard implementation of the language. Jython and IronPython implement Python programs for use in Java and .NET environments, respectively; they are alternative compilers for Python.

How You Run Programs

OK, it's time to start running some code. Now that you have a handle on program execution, you're finally ready to start some real Python programming. At this point, I'll assume that you have Python installed on your computer; if not, see the prior chapter and [Appendix A](#) for installation and configuration hints.

There are a variety of ways to tell Python to execute the code you type. This chapter discusses all the program launching techniques in common use today. Along the way, you'll learn how to type code *interactively* and how to save it in *files* to be run with system command lines, icon clicks, module imports and reloads, `exec` calls, menu options in GUIs such as IDLE, and more.

If you just want to find out how to run a Python program quickly, you may be tempted to read the parts of this chapter that pertain only to your platform and move on to [Chapter 4](#). But don't skip the material on module imports, as that's essential to understanding Python's program architecture. I also encourage you to at least skim the sections on IDLE and other IDEs, so you'll know what tools are available for when you start developing more sophisticated Python programs.

The Interactive Prompt

Perhaps the simplest way to run Python programs is to type them at Python's interactive command line, sometimes called the *interactive prompt*. There are a variety of ways to start this command line: in an IDE, from a system console, and so on. Assuming the interpreter is installed as an executable program on your system, the most platform-neutral way to start an interactive interpreter session is usually just to type **python** at your operating system's prompt, without any arguments. For example:

```
% python
Python 3.0.1 (r301:69561, Feb 13 2009, 20:04:18) [MSC v.1500 32 bit (Intel)] ...
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Typing the word “python” at your system shell prompt like this begins an interactive Python session; the “%” character at the start of this listing stands for a generic system prompt in this book—it’s not input that you type yourself. The notion of a system shell prompt is generic, but exactly how you access it varies by platform:

- On Windows, you can type **python** in a DOS console window (a.k.a. the Command Prompt, usually found in the Accessories section of the Start→Programs menu) or in the Start→Run... dialog box.
- On Unix, Linux, and Mac OS X, you might type this command in a shell or terminal window (e.g., in an *xterm* or console running a shell such as *ksh* or *csh*).
- Other systems may use similar or platform-specific devices. On handheld devices, for example, you generally click the Python icon in the home or application window to launch an interactive session.

If you have not set your shell’s **PATH** environment variable to include Python’s install directory, you may need to replace the word “python” with the full path to the Python executable on your machine. On Unix, Linux, and similar, **/usr/local/bin/python** or **/usr/bin/python** will often suffice. On Windows, try typing **C:\Python30\python** (for version 3.0):

```
C:\misc> c:\python30\python
Python 3.0.1 (r301:69561, Feb 13 2009, 20:04:18) [MSC v.1500 32 bit (Intel)] ...
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Alternatively, you can run a change-directory command to go to Python’s install directory before typing “python”—try the **cd c:\python30** command on Windows, for example:

```
C:\misc> cd C:\Python30
C:\Python30> python
Python 3.0.1 (r301:69561, Feb 13 2009, 20:04:18) [MSC v.1500 32 bit (Intel)] ...
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

On Windows, besides typing **python** in a shell window, you can also begin similar interactive sessions by starting IDLE’s main window (discussed later) or by selecting the “Python (command line)” menu option from the Start button menu for Python, as shown in [Figure 2-1](#) back in [Chapter 2](#). Both spawn a Python interactive prompt with equivalent functionality; typing a shell command isn’t necessary.

Running Code Interactively

However it's started, the Python interactive session begins by printing two lines of informational text (which I'll omit from most of this book's examples to save space), then prompts for input with `>>>` when it's waiting for you to type a new Python statement or expression. When working interactively, the results of your code are displayed after the `>>>` lines after you press the Enter key.

For instance, here are the results of two Python `print` statements (`print` is really a function call in Python 3.0, but not in 2.6, so the parentheses here are required in 3.0 only):

```
% python
>>> print('Hello world!')
Hello world!
>>> print(2 ** 8)
256
```

Again, you don't need to worry about the details of the `print` statements shown here yet; we'll start digging into syntax in the next chapter. In short, they print a Python string and an integer, as shown by the output lines that appear after each `>>>` input line (`2 ** 8` means 2 raised to the power 8 in Python).

When coding interactively like this, you can type as many Python commands as you like; each is run immediately after it's entered. Moreover, because the interactive session automatically prints the results of expressions you type, you don't usually need to say "print" explicitly at this prompt:

```
>>> lumberjack = 'okay'
>>> lumberjack
'okay'
>>> 2 ** 8
256
>>>
% <== Use Ctrl-D (on Unix) or Ctrl-Z (on Windows) to exit
```

Here, the first line saves a value by assigning it to a variable, and the last two lines typed are expressions (`lumberjack` and `2 ** 8`)—their results are displayed automatically. To exit an interactive session like this one and return to your system shell prompt, type Ctrl-D on Unix-like machines; on MS-DOS and Windows systems, type Ctrl-Z to exit. In the IDLE GUI discussed later, either type Ctrl-D or simply close the window.

Now, we didn't do much in this session's code—just typed some Python `print` and assignment statements, along with a few expressions, which we'll study in detail later. The main thing to notice is that the interpreter executes the code entered on each line immediately, when the Enter key is pressed.

For example, when we typed the first `print` statement at the `>>>` prompt, the output (a Python string) was echoed back right away. There was no need to create a source-code file, and no need to run the code through a compiler and linker first, as you'd normally do when using a language such as C or C++. As you'll see in later chapters, you can also run multiline statements at the interactive prompt; such a statement runs immediately after you've entered all of its lines and pressed Enter twice to add a blank line.

Why the Interactive Prompt?

The interactive prompt runs code and echoes results as you go, but it doesn't save your code in a file. Although this means you won't do the bulk of your coding in interactive sessions, the interactive prompt turns out to be a great place to both *experiment* with the language and *test* program files on the fly.

Experimenting

Because code is executed immediately, the interactive prompt is a perfect place to experiment with the language and will be used often in this book to demonstrate smaller examples. In fact, this is the first rule of thumb to remember: if you're ever in doubt about how a piece of Python code works, fire up the interactive command line and try it out to see what happens.

For instance, suppose you're reading a Python program's code and you come across an expression like `'Spam!' * 8` whose meaning you don't understand. At this point, you can spend 10 minutes wading through manuals and books to try to figure out what the code does, or you can simply run it interactively:

```
>>> 'Spam!' * 8                                     <== Learning by trying
'Spam!Spam!Spam!Spam!Spam!Spam!Spam!Spam!'
```

The immediate feedback you receive at the interactive prompt is often the quickest way to deduce what a piece of code does. Here, it's clear that it does string repetition: in Python `*` means multiply for numbers, but repeat for strings—it's like concatenating a string to itself repeatedly (more on strings in [Chapter 4](#)).

Chances are good that you won't break anything by experimenting this way—at least, not yet. To do real damage, like deleting files and running shell commands, you must really try, by importing modules explicitly (you also need to know more about Python's system interfaces in general before you will become that dangerous!). Straight Python code is almost always safe to run.

For instance, watch what happens when you make a mistake at the interactive prompt:

```
>>> X                                     <== Making mistakes
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'X' is not defined
```

In Python, using a variable before it has been assigned a value is always an error (otherwise, if names were filled in with defaults, some errors might go undetected). We'll learn more about that later; the important point here is that you don't crash Python or your computer when you make a mistake this way. Instead, you get a meaningful error message pointing out the mistake and the line of code that made it, and you can continue on in your session or script. In fact, once you get comfortable with Python, its error messages may often provide as much debugging support as you'll need (you'll read more on debugging in the sidebar [“Debugging Python Code” on page 67](#)).

Testing

Besides serving as a tool for experimenting while you're learning the language, the interactive interpreter is also an ideal place to test code you've written in files. You can import your module files interactively and run tests on the tools they define by typing calls at the interactive prompt.

For instance, of the following tests a function in a precoded module that ships with Python in its standard library (it prints the name of the directory you're currently working in), but you can do the same once you start writing module files of your own:

```
>>> import os
>>> os.getcwd()                             <== Testing on the fly
'c:\\Python30'
```

More generally, the interactive prompt is a place to test program components, regardless of their source—you can import and test functions and classes in your Python files, type calls to linked-in C functions, exercise Java classes under Jython, and more. Partly because of its interactive nature, Python supports an experimental and exploratory programming style you'll find convenient when getting started.

Using the Interactive Prompt

Although the interactive prompt is simple to use, there are a few tips that beginners should keep in mind. I'm including lists of common mistakes like this in this chapter for reference, but they might also spare you from a few headaches if you read them up front:

- **Type Python commands only.** First of all, remember that you can only type Python code at the Python prompt, not system commands. There are ways to run system commands from within Python code (e.g., with `os.system`), but they are not as direct as simply typing the commands themselves.

- **print statements are required only in files.** Because the interactive interpreter automatically prints the results of expressions, you do not need to type complete `print` statements interactively. This is a nice feature, but it tends to confuse users when they move on to writing code in files: within a code file, you must use `print` statements to see your output because expression results are not automatically echoed. Remember, you must say `print` in files, but not interactively.
- **Don't indent at the interactive prompt (yet).** When typing Python programs, either interactively or into a text file, be sure to start all your unnested statements in column 1 (that is, all the way to the left). If you don't, Python may print a "SyntaxError" message, because blank space to the left of your code is taken to be indentation that groups nested statements. Until [Chapter 10](#), all statements you write will be unnested, so this includes everything for now. This seems to be a recurring confusion in introductory Python classes. Remember, a leading space generates an error message.
- **Watch out for prompt changes for compound statements.** We won't meet *compound* (multiline) statements until [Chapter 4](#), and not in earnest until [Chapter 10](#), but as a preview, you should know that when typing lines 2 and beyond of a compound statement interactively, the prompt may change. In the simple shell window interface, the interactive prompt changes to `...` instead of `>>>` for lines 2 and beyond; in the IDLE interface, lines after the first are automatically indented. You'll see why this matters in [Chapter 10](#). For now, if you happen to come across a `...` prompt or a blank line when entering your code, it probably means that you've somehow confused interactive Python into thinking you're typing a multiline statement. Try hitting the Enter key or a Ctrl-C combination to get back to the main prompt. The `>>>` and `...` prompt strings can also be changed (they are available in the built-in module `sys`), but I'll assume they have not been in the book's example listings.
- **Terminate compound statements at the interactive prompt with a blank line.** At the interactive prompt, inserting a blank line (by hitting the Enter key at the start of a line) is necessary to tell interactive Python that you're done typing the multiline statement. That is, you must press Enter twice to make a compound statement run. By contrast, blank lines are not required in files and are simply ignored if present. If you don't press Enter twice at the end of a compound statement when working interactively, you'll appear to be stuck in a limbo state, because the interactive interpreter will do nothing at all—it's waiting for you to press Enter again!
- **The interactive prompt runs one statement at a time.** At the interactive prompt, you must run one statement to completion before typing another. This is natural for simple statements, because pressing the Enter key runs the statement entered. For compound statements, though, remember that you must submit a blank line to terminate the statement and make it run before you can type the next statement.

Entering multiline statements

At the risk of repeating myself, I received emails from readers who'd gotten burned by the last two points as I was updating this chapter, so it probably merits emphasis. I'll introduce multiline (a.k.a. compound) statements in the next chapter, and we'll explore their syntax more formally later in this book. Because their behavior differs slightly in files and at the interactive prompt, though, two cautions are in order here.

First, be sure to terminate multiline compound statements like `for` loops and `if` tests at the interactive prompt with a blank line. *You must press the Enter key twice*, to terminate the whole multiline statement and then make it run. For example (pun not intended...):

```
>>> for x in 'spam':  
...     print(x)  
...  
...  
                                     <== Press Enter twice here to make this loop run
```

You don't need the blank line after compound statements in a script file, though; this is required *only* at the interactive prompt. In a file, blank lines are not required and are simply ignored when present; at the interactive prompt, they terminate multiline statements.

Also bear in mind that the interactive prompt runs just *one statement at a time*: you must press Enter twice to run a loop or other multiline statement before you can type the next statement:

```
>>> for x in 'spam':  
...     print(x)  
...     print('done')  
...  
File "<stdin>", line 3  
    print('done')  
    ^  
SyntaxError: invalid syntax  
                                     <== Need to press Enter twice before a new statement
```

This means you can't cut and paste multiple lines of code into the interactive prompt, unless the code includes blank lines after each compound statement. Such code is better run in a file—the next section's topic.

System Command Lines and Files

Although the interactive prompt is great for experimenting and testing, it has one big disadvantage: programs you type there go away as soon as the Python interpreter executes them. Because the code you type interactively is never stored in a file, you can't run it again without retyping it from scratch. Cut-and-paste and command recall can help some here, but not much, especially when you start writing larger programs. To cut and paste code from an interactive session, you would have to edit out Python prompts, program outputs, and so on—not exactly a modern software development methodology!

To save programs permanently, you need to write your code in files, which are usually known as *modules*. Modules are simply text files containing Python statements. Once coded, you can ask the Python interpreter to execute the statements in such a file any number of times, and in a variety of ways—by system command lines, by file icon clicks, by options in the IDLE user interface, and more. Regardless of how it is run, Python executes all the code in a module file from top to bottom each time you run the file.

Terminology in this domain can vary somewhat. For instance, module files are often referred to as *programs* in Python—that is, a program is considered to be a series of precoded statements stored in a file for repeated execution. Module files that are run directly are also sometimes called *scripts*—an informal term usually meaning a top-level program file. Some reserve the term “module” for a file imported from another file. (More on the meaning of “top-level” and imports in a few moments.)

Whatever you call them, the next few sections explore ways to run code typed into module files. In this section, you’ll learn how to run files in the most basic way: by listing their names in a `python` command line entered at your computer’s system prompt. Though it might seem primitive to some, for many programmers a system shell command-line window, together with a text editor window, constitutes as much of an integrated development environment as they will ever need.

A First Script

Let’s get started. Open your favorite text editor (e.g., *vi*, Notepad, or the IDLE editor), and type the following statements into a new text file named *script1.py*:

```
# A first Python script
import sys                # Load a library module
print(sys.platform)
print(2 ** 100)           # Raise 2 to a power
x = 'Spam!'
print(x * 8)              # String repetition
```

This file is our first official Python script (not counting the two-liner in [Chapter 2](#)). You shouldn’t worry too much about this file’s code, but as a brief description, this file:

- Imports a Python module (libraries of additional tools), to fetch the name of the platform
- Runs three `print` function calls, to display the script’s results
- Uses a variable named `x`, created when it’s assigned, to hold onto a string object
- Applies various object operations that we’ll begin studying in the next chapter

The `sys.platform` here is just a string that identifies the kind of computer you’re working on; it lives in a standard Python module called `sys`, which you must import to load (again, more on imports later).

For color, I’ve also added some formal Python *comments* here—the text after the # characters. Comments can show up on lines by themselves, or to the right of code on a line. The text after a # is simply ignored as a human-readable comment and is not considered part of the statement’s syntax. If you’re copying this code, you can ignore the comments as well. In this book, we usually use a different formatting style to make comments more visually distinctive, but they’ll appear as normal text in your code.

Again, don’t focus on the syntax of the code in this file for now; we’ll learn about all of it later. The main point to notice is that you’ve typed this code into a file, rather than at the interactive prompt. In the process, you’ve coded a fully functional Python script.

Notice that the module file is called *script1.py*. As for all top-level files, it could also be called simply *script*, but files of code you want to *import* into a client have to end with a *.py* suffix. We’ll study imports later in this chapter. Because you may want to import them in the future, it’s a good idea to use *.py* suffixes for most Python files that you code. Also, some text editors detect Python files by their *.py* suffix; if the suffix is not present, you may not get features like syntax colorization and automatic indentation.

Running Files with Command Lines

Once you’ve saved this text file, you can ask Python to run it by listing its full filename as the first argument to a `python` command, typed at the system shell prompt:

```
% python script1.py
win32
1267650600228229401496703205376
Spam! Spam! Spam! Spam! Spam! Spam! Spam!
```

Again, you can type such a system shell command in whatever your system provides for command-line entry—a Windows Command Prompt window, an xterm window, or similar. Remember to replace “python” with a full directory path, as before, if your `PATH` setting is not configured.

If all works as planned, this shell command makes Python run the code in this file line by line, and you will see the output of the script’s three `print` statements—the name of the underlying platform, 2 raised to the power 100, and the result of the same string repetition expression we saw earlier (again, more on the last two of these in [Chapter 4](#)).

If all didn’t work as planned, you’ll get an error message—make sure you’ve entered the code in your file exactly as shown, and try again. We’ll talk about debugging options in the sidebar “[Debugging Python Code](#)” on page 67, but at this point in the book your best bet is probably rote imitation.

Because this scheme uses shell command lines to start Python programs, all the usual shell syntax applies. For instance, you can route the output of a Python script to a file to save it for later use or inspection by using special shell syntax:

```
% python script1.py > saveit.txt
```

In this case, the three output lines shown in the prior run are stored in the file *saveit.txt* instead of being printed. This is generally known as *stream redirection*; it works for input and output text and is available on Windows and Unix-like systems. It also has little to do with Python (Python simply supports it), so we will skip further details on shell redirection syntax here.

If you are working on a Windows platform, this example works the same, but the system prompt is normally different:

```
C:\Python30> python script1.py
win32
1267650600228229401496703205376
Spam! Spam! Spam! Spam! Spam! Spam! Spam! Spam! Spam!
```

As usual, be sure to type the full path to Python if you haven't set your `PATH` environment variable to include this path or run a change-directory command to go to the path:

```
D:\temp> C:\python30\python script1.py
win32
1267650600228229401496703205376
Spam! Spam! Spam! Spam! Spam! Spam! Spam! Spam!
```

On all recent versions of Windows, you can also type just the name of your script, and omit the name of Python itself. Because newer Windows systems use the Windows Registry to find a program with which to run a file, you don't need to name "python" on the command line explicitly to run a *.py* file. The prior command, for example, could be simplified to this on most Windows machines:

```
D:\temp> script1.py
```

Finally, remember to give the full path to your script file if it lives in a different directory from the one in which you are working. For example, the following system command line, run from *D:\other*, assumes Python is in your system path but runs a file located elsewhere:

```
D:\other> python c:\code\otherscript.py
```

If your `PATH` doesn't include Python's directory, and neither Python nor your script file is in the directory you're working in, use full paths for both:

```
D:\other> C:\Python30\python c:\code\otherscript.py
```

Using Command Lines and Files

Running program files from system command lines is also a fairly straightforward launch option, especially if you are familiar with command lines in general from prior work. For newcomers, though, here are a few pointers about common beginner traps that might help you avoid some frustration:

- **Beware of automatic extensions on Windows.** If you use the Notepad program to code program files on Windows, be careful to pick the type All Files when it comes time to save your file, and give the file a `.py` suffix explicitly. Otherwise, Notepad will save your file with a `.txt` extension (e.g., as *script1.py.txt*), making it difficult to run in some launching schemes.

Worse, Windows hides file extensions by default, so unless you have changed your view options you may not even notice that you’ve coded a text file and not a Python file. The file’s icon may give this away—if it doesn’t have a snake on it, you may have trouble. Uncolored code in IDLE and files that open to edit instead of run when clicked are other symptoms of this problem.

Microsoft Word similarly adds a `.doc` extension by default; much worse, it adds formatting characters that are not legal Python syntax. As a rule of thumb, always pick All Files when saving under Windows, or use a more programmer-friendly text editor such as IDLE. IDLE does not even add a `.py` suffix automatically—a feature programmers tend to like, but users do not.

- **Use file extensions and directory paths at system prompts, but not for imports.** Don’t forget to type the full name of your file in system command lines—that is, use `python script1.py` rather than `python script1`. By contrast, Python’s `import` statements, which we’ll meet later in this chapter, omit both the `.py` file suffix and the directory path (e.g., `import script1`). This may seem trivial, but confusing these two is a common mistake.

At the system prompt, you are in a system shell, not Python, so Python’s module file search rules do not apply. Because of that, you must include both the `.py` extension and, if necessary, the full directory path leading to the file you wish to run. For instance, to run a file that resides in a different directory from the one in which you are working, you would typically list its full path (e.g., `python d:\tests\spam.py`). Within Python code, however, you can just say `import spam` and rely on the Python module search path to locate your file, as described later.

- **Use print statements in files.** Yes, we’ve already been over this, but it is such a common mistake that it’s worth repeating at least once here. Unlike in interactive coding, you generally must use `print` statements to see output from program files. If you don’t see any output, make sure you’ve said “print” in your file. Again, though, `print` statements are *not* required in an interactive session, since Python automatically echoes expression results; `prints` don’t hurt here, but are superfluous extra typing.

Unix Executable Scripts (#!)

If you are going to use Python on a Unix, Linux, or Unix-like system, you can also turn files of Python code into executable programs, much as you would for programs coded in a shell language such as *bash* or *ksh*. Such files are usually called *executable scripts*. In simple terms, Unix-style executable scripts are just normal text files containing Python statements, but with two special properties:

- **Their first line is special.** Scripts usually start with a line that begins with the characters `#!` (often called “hash bang”), followed by the path to the Python interpreter on your machine.
- **They usually have executable privileges.** Script files are usually marked as executable to tell the operating system that they may be run as top-level programs. On Unix systems, a command such as `chmod +x file.py` usually does the trick.

Let’s look at an example for Unix-like systems. Use your text editor again to create a file of Python code called *brian*:

```
#!/usr/local/bin/python
print('The Bright Side ' + 'of Life...')           # + means concatenate for strings
```

The special line at the top of the file tells the system where the Python interpreter lives. Technically, the first line is a Python comment. As mentioned earlier, all comments in Python programs start with a `#` and span to the end of the line; they are a place to insert extra information for human readers of your code. But when a comment such as the first line in this file appears, it’s special because the operating system uses it to find an interpreter for running the program code in the rest of the file.

Also, note that this file is called simply *brian*, without the *.py* suffix used for the module file earlier. Adding a *.py* to the name wouldn’t hurt (and might help you remember that this is a Python program file), but because you don’t plan on letting other modules import the code in this file, the name of the file is irrelevant. If you give the file executable privileges with a `chmod +x brian` shell command, you can run it from the operating system shell as though it were a binary program:

```
% brian
The Bright Side of Life...
```

A note for Windows users: the method described here is a Unix trick, and it may not work on your platform. Not to worry; just use the basic command-line technique explored earlier. List the file’s name on an explicit `python` command line:*

* As we discussed when exploring command lines, modern Windows versions also let you type just the name of a *.py* file at the system command line—they use the Registry to determine that the file should be opened with Python (e.g., typing `brian.py` is equivalent to typing `python brian.py`). This command-line mode is similar in spirit to the Unix `#!`, though it is system-wide on Windows, not per-file. Note that some programs may actually interpret and use a first `#!` line on Windows much like on Unix, but the DOS system shell on Windows simply ignores it.

```
C:\misc> python brian
The Bright Side of Life...
```

In this case, you don't need the special `#!` comment at the top (although Python just ignores it if it's present), and the file doesn't need to be given executable privileges. In fact, if you want to run files portably between Unix and Microsoft Windows, your life will probably be simpler if you always use the basic command-line approach, not Unix-style scripts, to launch programs.

The Unix `env` Lookup Trick

On some Unix systems, you can avoid hardcoding the path to the Python interpreter by writing the special first-line comment like this:

```
#!/usr/bin/env python
...script goes here...
```

When coded this way, the `env` program locates the Python interpreter according to your system search path settings (i.e., in most Unix shells, by looking in all the directories listed in the `PATH` environment variable). This scheme can be more portable, as you don't need to hardcode a Python install path in the first line of all your scripts.

Provided you have access to `env` everywhere, your scripts will run no matter where Python lives on your system—you need only change the `PATH` environment variable settings across platforms, not in the first line in all your scripts. Of course, this assumes that `env` lives in the same place everywhere (on some machines, it may be in `/sbin`, `/bin`, or elsewhere); if not, all portability bets are off!

Clicking File Icons

On Windows, the Registry makes opening files with icon clicks easy. Python automatically registers itself to be the program that opens Python program files when they are clicked. Because of that, it is possible to launch the Python programs you write by simply clicking (or double-clicking) on their file icons with your mouse cursor.

On non-Windows systems, you will probably be able to perform a similar trick, but the icons, file explorer, navigation schemes, and more may differ slightly. On some Unix systems, for instance, you may need to register the `.py` extension with your file explorer GUI, make your script executable using the `#!` trick discussed in the previous section, or associate the file MIME type with an application or command by editing files, installing programs, or using other tools. See your file explorer's documentation for more details if clicks do not work correctly right off the bat.

Clicking Icons on Windows

To illustrate, let's keep using the script we wrote earlier, `script1.py`, repeated here to minimize page flipping:

```

# A first Python script
import sys                # Load a library module
print(sys.platform)
print(2 ** 100)           # Raise 2 to a power
x = 'Spam!'
print(x * 8)              # String repetition

```

As we've seen, you can always run this file from a system command line:

```

C:\misc> c:\python30\python script1.py
win32
1267650600228229401496703205376

```

However, icon clicks allow you to run the file without any typing at all. If you find this file's icon—for instance, by selecting Computer (or My Computer in XP) in your Start menu and working your way down on the C drive on Windows—you will get the file explorer picture captured in [Figure 3-1](#) (Windows Vista is being used here). Python source files show up with white backgrounds on Windows, and byte code files show up with black backgrounds. You will normally want to click (or otherwise run) the source code file, in order to pick up your most recent changes. To launch the file here, simply click on the icon for *script1.py*.

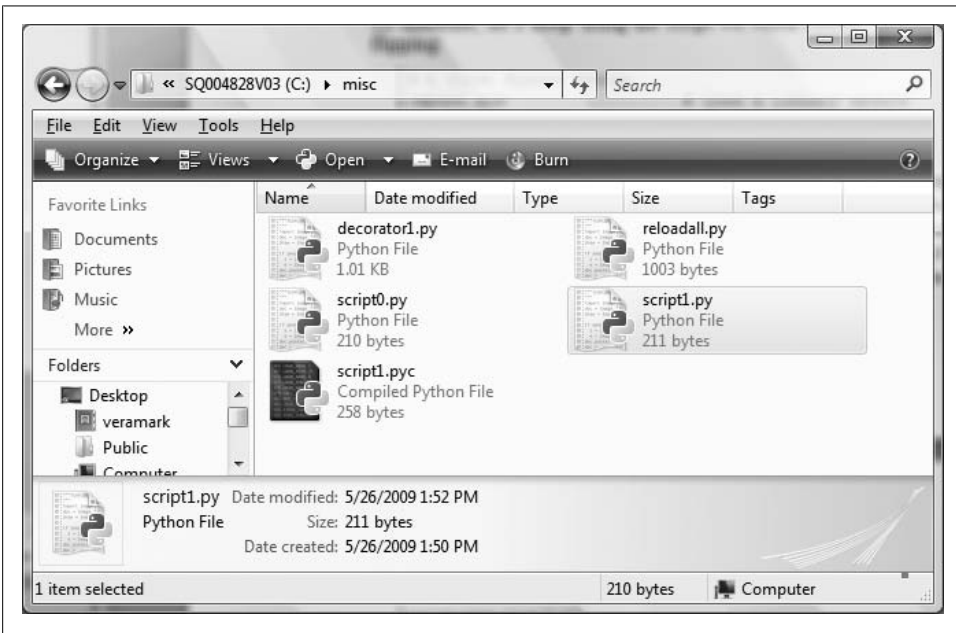


Figure 3-1. On Windows, Python program files show up as icons in file explorer windows and can automatically be run with a double-click of the mouse (though you might not see printed output or error messages this way).

The input Trick

Unfortunately, on Windows, the result of clicking on a file icon may not be incredibly satisfying. In fact, as it is, this example script generates a perplexing “flash” when clicked—not exactly the sort of feedback that budding Python programmers usually hope for! This is not a bug, but has to do with the way the Windows version of Python handles printed output.

By default, Python generates a pop-up black DOS console window to serve as a clicked file’s input and output. If a script just prints and exits, well, it just prints and exits—the console window appears, and text is printed there, but the console window closes and disappears on program exit. Unless you are very fast, or your machine is very slow, you won’t get to see your output at all. Although this is normal behavior, it’s probably not what you had in mind.

Luckily, it’s easy to work around this. If you need your script’s output to stick around when you launch it with an icon click, simply put a call to the built-in `input` function at the very bottom of the script (`raw_input` in 2.6: see the note ahead). For example:

```
# A first Python script
import sys                # Load a library module
print(sys.platform)
print(2 ** 100)           # Raise 2 to a power
x = 'Spam!'
print(x * 8)              # String repetition
input()                   # <== ADDED
```

In general, `input` reads the next line of standard input, waiting if there is none yet available. The net effect in this context will be to pause the script, thereby keeping the output window shown in [Figure 3-2](#) open until you press the Enter key.



Figure 3-2. When you click a program’s icon on Windows, you will be able to see its printed output if you include an `input` call at the very end of the script. But you only need to do so in this context!